

PS Rental Cloud Vault

Manual Penggunaan Aplikasi

Sistem Rental PlayStation 3 dengan Manajemen AWS S3 (Cloud Vault)

Disusun untuk Tugas Cloud Computing

Universitas Islam Sultan Agung (UNISSULA)

2026

Daftar Isi

1. Pendahuluan
2. Konsep Aplikasi: Rental PS3 + Manajemen S3
3. Fitur Utama Aplikasi
4. Arsitektur & Struktur Folder
5. Persiapan Akun AWS (IAM & Kredensial)
6. Instalasi & Menjalankan Aplikasi
7. Gambaran Antarmuka (Layout)
8. Alur Penggunaan Aplikasi (End-to-End)
9. Troubleshooting
10. Penutup

1. Pendahuluan

PS Rental Cloud Vault adalah aplikasi web manajemen rental PlayStation 3 yang dibangun menggunakan Python (Flask), SQLite untuk data transaksional, dan AWS S3 (lewat boto3) sebagai media penyimpanan dokumentasi setiap transaksi sewa. Aplikasi ini dikembangkan untuk tugas mata kuliah Cloud Computing yang mensyaratkan aplikasi fungsional untuk memanajemen layanan S3 atau EC2.

Berbeda dari aplikasi file manager S3 generik, di sini S3 diberi konteks bisnis nyata: setiap transaksi sewa PS3 otomatis mendapat folder penyimpanan sendiri di S3 (disebut 'Cloud Vault'), tempat admin dapat mengunggah, melihat, mengganti nama, dan menghapus foto kondisi unit serta bukti pembayaran — persis seperti file manager S3 pada umumnya, namun terintegrasi penuh dengan alur bisnis rental.

2. Konsep Aplikasi: Rental PS3 + Manajemen S3

Aplikasi terbagi menjadi dua sisi yang saling terhubung:

Sisi Bisnis (Rental)	Sisi Cloud (S3 Management)
Data unit PS3, pelanggan, dan transaksi sewa disimpan di database SQLite lokal.	Foto kondisi unit, bukti pembayaran disimpan di AWS S3, dikelola penuh oleh aplikasi.
Setiap transaksi baru dibuat lewat form sederhana (pid, tipe, tanggal, lokasi).	Setiap transaksi otomatis mendapat folder S3 sendiri: rentals/{id}-{kode_unit}.
Halaman detail transaksi menampilkan ringkasan biaya & status sewa.	Dari halaman detail transaksi, admin bisa langsung 'Buka Folder Vault' untuk akses file.

3. Fitur Utama Aplikasi

3.1 Manajemen Bisnis Rental

Fitur	Deskripsi
Dashboard	Statistik unit, penyewaan aktif, pelanggan, pendapatan, storage vault
Manajemen Unit PS3	Tambah unit, ubah status (Tersedia/Disewa/Maintenance), hapus
Manajemen Pelanggan	Data pelanggan: nama, telepon, email, alamat
Transaksi Sewa	Buat transaksi baru, hitung otomatis total biaya, tandai dikembalikan

3.2 Cloud Vault — Modul Manajemen S3

Fitur	Deskripsi
Folder Otomatis per Transaksi	Setiap transaksi baru otomatis dibuatkan folder S3 tersendiri
Browse File/Folder	Navigasi folder virtual S3 dengan breadcrumb
Upload	Drag & drop atau pilih file, multi-file sekaligus
Download	Lewat presigned URL yang aman & kedaluwarsa otomatis
Delete	File tunggal maupun folder beserta isinya (rekursif)
Rename	File maupun folder (implementasi copy + delete di S3)
Create Folder	Folder tambahan, misalnya per kategori dokumen
Preview Gambar	Melihat isi file gambar langsung tanpa mengunduhnya

4. Arsitektur & Struktur Folder

Aplikasi disusun modular memakai pola Blueprint Flask, memisahkan routing bisnis rental (routes/units.py, customers.py, rentals.py), modul manajemen S3 (routes/vault.py), layer service (services/), dan database (database/models.py).

```
ps-rental-cloud-vault/
■■■■ app.py                                (entry point Flask)
■■■■ config.py                            (konfigurasi & environment variable)
■■■■ requirements.txt
■■■■ Dockerfile / docker-compose.yml
■■■■ static/css, static/js                (tampilan tema "Neon Console")
■■■■ templates/                          (halaman HTML/Jinja2)
■■■■ routes/
■    ■■■■ dashboard.py
■    ■■■■ units.py                        (CRUD unit PS3)
■    ■■■■ customers.py                   (CRUD pelanggan)
■    ■■■■ rentals.py                     (transaksi sewa + trigger folder S3)
■    ■■■■ vault.py                       (semua endpoint manajemen S3)
■■■■ services/
■    ■■■■ s3_service.py                  (semua logic boto3 terpusat)
■    ■■■■ db_service.py                  (query database rental)
■■■■ database/models.py                   (skema SQLite + seed data)
■■■■ utils/helper.py
■■■■ docs/Manual.pdf                     (dokumen ini)
```

Relasi kunci: tabel **rentals** pada SQLite menyimpan kolom *vault_prefix* yang berisi path folder S3 milik transaksi tersebut, sehingga halaman detail transaksi bisa langsung mengarahkan admin ke folder yang tepat di Cloud Vault.

5. Persiapan Penyimpanan: LocalStack (S3 Lokal)

Aplikasi ini secara default memakai **LocalStack**, emulator AWS S3 yang berjalan penuh di komputer sendiri lewat Docker — tidak membutuhkan akun AWS asli maupun biaya apa pun. Kode aplikasi tetap memakai boto3 asli; hanya alamat endpoint API yang diarahkan ke LocalStack. Jika suatu saat dibutuhkan, aplikasi bisa dipindah ke AWS S3 sungguhan hanya dengan mengubah isi file .env, tanpa mengubah kode sama sekali.

- 1 Pastikan Docker sudah terpasang di komputer.
- 2 Jalankan LocalStack: `docker run -d --name localstack -p 4566:4566 -e SERVICES=s3 localstack/localstack:3`
- 3 Salin **.env.example** menjadi **.env** — secara default sudah diarahkan ke LocalStack (`AWS_ENDPOINT_URL=http://localhost:4566`), kredensial cukup diisi 'test'/'test'.
- 4 Tentukan nama bucket pada **VAULT_BUCKET** — aplikasi otomatis membuat bucket ini di LocalStack saat pertama kali dijalankan.
- 5 (Opsional, untuk produksi) Untuk pindah ke AWS S3 asli: buat IAM User dengan permission **AmazonS3FullAccess**, isi kredensial asli di .env, lalu kosongkan `AWS_ENDPOINT_URL` — lihat README.md untuk detail policy.

6. Instalasi & Menjalankan Aplikasi

6.1 Menjalankan Secara Lokal (Python + LocalStack terpisah)

```
git clone https://github.com/username-anda/ps-rental-cloud-vault.git
cd ps-rental-cloud-vault
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt

docker run -d --name localstack -p 4566:4566 -e SERVICES=s3 localstack/localstack:3
cp .env.example .env # default sudah mengarah ke LocalStack
python app.py
```

Database SQLite (rental.db) otomatis dibuat dengan 3 unit PS3 contoh, dan bucket vault otomatis dibuat di LocalStack saat aplikasi pertama kali dijalankan. Buka **http://localhost:5000**.

6.2 Menjalankan Sepenuhnya dengan Docker Compose (app + LocalStack)

```
cp .env.example .env # default sudah cocok untuk mode Docker+LocalStack
touch rental.db
docker compose up --build
```

docker-compose.yml sudah berisi dua service: **localstack** (emulator S3) dan **app** (aplikasi Flask), otomatis saling terhubung dalam satu network Docker.

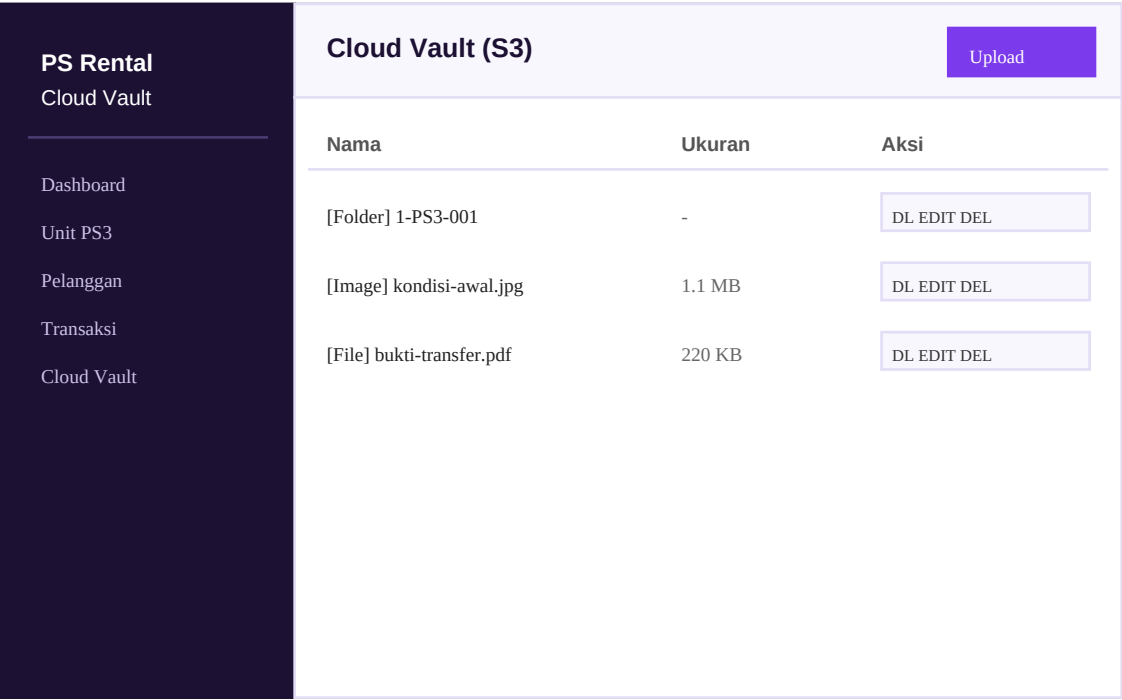
6.3 Upload Kode ke GitHub

```
git init
git add .
git commit -m "Initial commit - PS Rental Cloud Vault"
git branch -M main
git remote add origin https://github.com/username-anda/ps-rental-cloud-vault.git
git push -u origin main
```

File **.env** dan **rental.db** sudah dikecualikan lewat .gitignore supaya kredensial rahasia tidak ikut ter-commit.

7. Gambaran Antarmuka (Layout)

Antarmuka menggunakan tema "Neon Console" (latar terang, aksen ungu & hijau) dengan tiga area utama: sidebar navigasi gelap di kiri, topbar berisi judul halaman & tombol aksi, dan area konten utama.



Gambar 1. Skema layout halaman Cloud Vault

8. Alur Penggunaan Aplikasi (End-to-End)

8.1 Menyiapkan Data Master

- 1 Buka menu **Unit PS3**, klik **+ Tambah Unit**, isi kode, nama, kondisi, dan harga sewa per hari.
- 2 Buka menu **Pelanggan**, klik **+ Tambah Pelanggan**, isi nama dan nomor telepon.

8.2 Membuat Transaksi Sewa Baru

- 1 Klik **+ Transaksi Sewa Baru** dari dashboard atau menu Transaksi Sewa.
- 2 Pilih unit PS3 (hanya unit berstatus Tersedia yang muncul) dan pelanggan.
- 3 Isi tanggal mulai dan tanggal selesai sewa; total biaya dihitung otomatis.
- 4 Klik **Buat Transaksi**. Sistem otomatis: (a) menyimpan transaksi, (b) mengubah status unit menjadi Disewa, dan (c) membuat folder khusus di Cloud Vault S3 untuk transaksi ini.

8.3 Mengelola Dokumen Transaksi di Cloud Vault

- 1 Pada halaman detail transaksi, klik **Buka Folder Vault**.
- 2 Klik **Upload**, lalu unggah foto kondisi unit sebelum disewa dan bukti pembayaran.
- 3 Klik nama file gambar untuk membuka preview langsung di browser.
- 4 Gunakan ikon Rename/Hapus untuk mengelola file, atau **+ Folder** untuk membuat sub-folder tambahan (misalnya folder 'kondisi-akhir' saat unit dikembalikan).

8.4 Menyelesaikan Transaksi

- 1 Setelah pelanggan mengembalikan unit, buka halaman detail transaksi.
- 2 Klik **Tandai Dikembalikan**. Status transaksi berubah menjadi Selesai dan status unit otomatis kembali menjadi Tersedia.
- 3 Dokumentasi foto kondisi akhir dapat diunggah ke folder vault yang sama sebagai bukti serah terima.

8.5 Memantau Melalui Dashboard

- 1 Dashboard menampilkan jumlah unit tersedia/disewa, total pelanggan, pendapatan dari transaksi yang sudah selesai, serta total penggunaan storage Cloud Vault (S3).

9. Troubleshooting

Masalah	Kemungkinan Penyebab & Solusi
Dashboard error 'Cloud Vault belum terbentuk'	Pastikan container LocalStack sudah jalan (docker ps), dan AWS_ENDPOINT_URL di .env sudah benar.
Error 'Access Denied' saat upload/hapus file	Pastikan IAM policy user memiliki permission s3:PutObject, s3:DeleteObject, s3:GetObject.
Folder vault tidak terbentuk saat transaksi selesai	Cek permission s3:CreateBucket/HeadBucket, atau buat bucket VAULT_BUCKET secara manual.
Unit tidak muncul di form transaksi baru	Form hanya menampilkan unit berstatus 'Tersedia'. Ubah status unit lewat menu Unit PS3.
Preview gambar tidak muncul	Pastikan file berformat gambar dan presigned URL belum kedaluwarsa (default 10 menit).
Port 5000 sudah dipakai aplikasi lain	Ubah port pada app.run() di app.py atau hentikan aplikasi lain yang memakai port tersebut.

10. Penutup

PS Rental Cloud Vault menggabungkan sistem manajemen bisnis rental sehari-hari dengan modul manajemen AWS S3 yang fungsional dan terintegrasi penuh ke dalam alur kerja transaksi. Untuk detail teknis tambahan (struktur kode, environment variable, konfigurasi Docker) silakan merujuk pada README.md yang disertakan bersama source code proyek ini.